

CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 7

Topic : **Paging (MRU Page Replacement)**

Presented by,

Fahad Hasan Mahi
Reg : 23101186
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

Consider the following page reference string: **[1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]**.

Implement the **MRU (Most Recently Used)** page replacement algorithm with a **Frame Size of 4**. MRU operates on the logic that the page which was *most recently used* is the least likely to be used in the immediate future, so it replaces that specific page when a fault occurs.

Calculate the total Page Faults, Page Hits, and Page Hit Ratio for MRU.

Source Code :

```
def mru_page_replacement(reference_string, frame_size):
 frames = []
 last_used = {}
 page_faults = 0
 page_hits = 0

 print(f"MRU Page Replacement (Frame Size = {frame_size})")
 print(f"Reference String: {reference_string}\n")

 print("Step\tPage\tFrames\t\tStatus")
 print("-" * 45)

 for i, page in enumerate(reference_string):
 last_used[page] = i

 if page in frames:
 page_hits += 1
 status = "Hit"
 else:
 page_faults += 1
 status = "Fault"
 if len(frames) < frame_size:
 frames.append(page)
 else:
 victim = max(frames, key=lambda p: last_used[p])
 frames[frames.index(victim)] = page

 print(f"{i+1}\t{page}\t{frames}\t\t{status}")

 print("-" * 45)
```

```
print(f"Total Page Faults: {page_faults}")


print(f"Total Page Hits: {page_hits}")
print(f"Page Hit Ratio: {round(page_hits / len(reference_string), 2)}")

reference_string = [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
frame_size = 4

mru_page_replacement(reference_string, frame_size)
```

Output Screenshot :

```
❯ /usr/bin/python "/home/fahad369/OS-Lab/CSE_402_Assignmen
MRU Page Replacement (Frame Size = 4)
Reference String: [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]

Step    Page    Frames          Status
-----------------------------------------
1       1       [1]             Fault
2       2       [1, 2]          Fault
3       3       [1, 2, 3]               Fault
4       4       [1, 2, 3, 4]            Fault
5       1       [1, 2, 3, 4]            Hit
6       2       [1, 2, 3, 4]            Hit
7       5       [1, 5, 3, 4]            Fault
8       1       [1, 5, 3, 4]            Hit
9       2       [2, 5, 3, 4]            Fault
10      3       [2, 5, 3, 4]            Hit
11      4       [2, 5, 3, 4]            Hit
12      5       [2, 5, 3, 4]            Hit
-----------------------------------------
Total Page Faults: 6
Total Page Hits: 6
Page Hit Ratio: 0.5
```

Fig : MRU Page replacement

Comparison between FIFO vs MRU:

Numeric Comparison (Same Dataset, Frame Size = 4):

Metric	FIFO	MRU
Total Page Faults	10	6
Total Page Hits	2	6
Page Hit Ratio	0.17	0.50
Replacement Logic	Evicts the oldest page (First-In)	Evicts the most recently used page

Feature Comparison:

Feature	FIFO	MRU
Selection Criterion	Order of arrival in memory	Recency of use (highest timestamp)
Implementation Complexity	Very simple (circular pointer)	Moderate (requires tracking usage time)
Belady's Anamoly	Susceptible	Susceptible
Best Use	General purpose, simple systems	Specific access patterns where recently used pages won't be needed immediately
Predictibility	Highly predictable	Highly dependent on the specific reference string

Discussion :

The simulation results highlight how drastically the replacement logic impacts performance, even with the exact same memory size and request sequence. **MRU (Most Recently Used)** operates on the assumption that a page accessed very recently has just finished its current task and is less likely to be needed immediately again.

On this specific dataset, MRU significantly outperformed FIFO, achieving a **50% hit ratio** compared to FIFO's **17%**. When page 5 arrived at step 7, FIFO blindly evicted the oldest page (1), which was needed again at step 8, causing an immediate fault. MRU, however, correctly identified that page 2 was the *most recently used* at that exact moment and evicted it instead. Since page 2 wasn't needed until step 9, MRU successfully kept the frequently accessed pages (1, 3, 4) in memory, resulting in 4 extra hits.

However, it is important to note that MRU's success here is highly dependent on the access pattern. Unlike algorithms that track long-term frequency (like LFU), MRU only looks at the immediate past. If a reference string repeatedly accesses the same two pages in an alternating pattern (e.g., 1, 2, 1, 2, 1, 2), MRU will fail catastrophically by constantly evicting the page it just used, resulting in a 0% hit ratio.

Conclusion :

This lab demonstrated that while FIFO is simple and easy to implement, its ignorance of page usage patterns makes it highly inefficient for many real-world workloads, as proven by its low hit ratio (0.17) and susceptibility to Belady's Anomaly. MRU offers a different heuristic by evicting the most recently used page, which in this specific dataset doubled the hit ratio to 0.50 by preserving pages that were about to be reused.
Ultimately, neither algorithm is universally optimal. FIFO fails because it treats all pages equally regardless of use, while MRU fails when access patterns are highly repetitive. This reinforces why modern operating systems rely on more robust algorithms like LRU (Least Recently Used) or Clock algorithms, which balance recency and frequency to adapt to a wider variety of memory access behaviors.

Github repository : https://github.com/fhmahi957/CSE_402_Assignment_materials.git